

Thema: Assembler Programm mit I/O - Stoppuhr

Letzter Abgabetermin: Vor dem **neunten** Praktikumstermin via `git` über das Tag `Stoppuhr.i` (i ist eine fortlaufende Nummer, wobei das Tag mit dem höchsten i die gültige Abgabe kennzeichnet). Arbeiten Sie in Ihrem Team auf einem Repo.

Die Aufgabe ist über drei Wochen gestreckt. Laden Sie in jeder Woche die von Ihnen bearbeiteten Daten auf den `git` Server hoch. **Arbeiten Sie in jeder Woche an der Aufgabe**, ansonsten gefährden Sie den Abgabetermin. Es ist sinnvoll, wenn Sie

- in der ersten Woche Taster, LEDs und Display bearbeiten und testen.
- in der zweiten Woche die FSM (Finite State Machine) und die Zeitfunktionen bearbeiten und testen. Unterteilen Sie weiterhin das Programm in kleine Unterprogramme.
- in der dritten Woche alles zusammenfügen und intensiv testen.

Aufgabe:¹ Es ist eine **Stoppuhr** zu programmieren. **Die Darstellung der Zeit soll auf dem TFT-Display mit einer Auflösung von 1/100 Sekunden erfolgen.** Für die Steuerung der Uhr stehen die Taster S5, S6 und S7 zur Verfügung. Über die LEDs D8 und D9 wird der Betriebszustand der Uhr angezeigt.

Die Uhr hat drei Betriebszustände:

- **INIT** Die Uhrzeit ist zurückgesetzt, auf dem TFT wird die Zeit "00:00.00" angezeigt. Durch Betätigung der Taste S7 wechselt die Uhr in den Betriebszustand **RUNNING**.
- **RUNNING** Die Zeit, die seit dem Start vergangen ist, wird gemessen **und auf dem Display im Format "mm:ss.nn" angezeigt**, wobei mm die Minuten, ss die Sekunden und nn die hundertstel Sekunden darstellen. Durch Betätigung von S6 wechselt die Uhr in den Zustand **HOLD** und durch S5 in den Zustand **INIT**.
- **HOLD** Dieser Zustand dient zum Anzeigen der gestoppten Zeit zum Zeitpunkt der Betätigung von S6. Im Hintergrund läuft die Uhr weiter, so dass durch Drücken von S7 wieder die aktuelle Zeit angezeigt wird und die Uhr in den Zustand **RUNNING** wechselt. Durch Drücken von S5 wird die Uhrzeit gelöscht, d. h. die Uhr wechselt in den Betriebszustand **INIT**.

Die LEDs haben folgende Bedeutung

- **LED D8** leuchtet, wenn die Zeitmessung aktiv ist, also die Uhr in den Zuständen **RUNNING** oder **HOLD** ist.
- **LED D9** zeigt an, dass die Zeitanzeige angehalten wurde. Sie leuchtet somit im Zustand **HOLD**.

¹ Diese Aufgabe basiert auf Unterlagen zum GTP Praktikum von Prof. Dr. Heiner Heitmann, HAW Hamburg.

Initialisierung der Hardware Zum Programmstart werden mehrere Komponenten initialisiert. Den entsprechenden Code finden Sie im beigefügten Assembler Programm.

Ansteuerung der LEDs Zum Ein- und Ausschalten der LEDs existieren zwei 16 Bit Register mit den Adressen `GPIO_D_SET` und `GPIO_D_CLR`. LEDs können durch Schreiben auf die Adresse `GPIO_D_SET` eingeschaltet werden. Jeder LED ist ein Bit zugeordnet: LED D8: Bit 0, LED D9: Bit 1 usw. Beim Schreiben auf die Adresse `GPIO_G_SET` werden alle LEDs, deren Bits gesetzt sind, eingeschaltet. Alle anderen LEDs ändern ihren Zustand nicht. Das Ausschalten erfolgt durch das Schreiben auf die Adresse `GPIO_D_CLR`: Alle LEDs, deren Bits gesetzt sind, werden ausgeschaltet, der Zustand der anderen ändert sich nicht. Über die Adresse `GPIO_D_PIN` können Sie ein 16 Bit Register auslesen, das den aktuellen Zustand der LEDs enthält (s. beigefügtes Assembler Programm).

Abfrage der Taster Der Zustand der Taster lässt sich durch Lesen des 16 Bit Registers mit der Adresse `GPIO_F_PIN` abfragen. Jedem Taster ist ein Bit zugeordnet: S0: Bit 0, S1: Bit 1, usw. Ist in dem von der Adresse `GPIO_F_PIN` gelesenen Halbwort das entsprechende Bit gesetzt, so ist der Taster **nicht** gedrückt. Ist das Bit hingegen gelöscht, ist der Taster gedrückt (s. beigefügtes Assembler Programm).

Ansteuerung der TFT-Anzeige Die Anzeige kann über folgende Funktionen angesteuert werden:

- Die Funktion `lcdGoToXY` positioniert den Cursor **auf dem Display. Im Register R0 wird die X Position übergeben. Im Register R1 wird die Y Position übergeben.**
- Die Funktion `lcdPrintS` gibt an der Position des Cursors einen String auf dem Bildschirm aus. Die Adresse des Strings wird im Register R0 übergeben. Nachdem der String gedruckt wurde, wird die Zeile bis zum Rand des Displays mit Leerzeichen weiter bedruckt.
- Die Funktion `lcdPrintC` gibt an der aktuellen Position des Cursors ein Zeichen auf dem Display aus. Das Zeichen wird als ASCII Wert im Register R0 übergeben.

Der ISA Befehl `BL` springt eine Funktion an, so dass sie ausgeführt wird. Nach Abarbeitung der Funktion wird der Befehl hinter dem `BL` Befehl ausgeführt. Es ist möglich, dass nach dem Aufruf einer Funktion die Werte der Register `R0`, `R1`, `R2` und `R3` sich geändert haben. Ein Beispiel ist `BL initITSboard` am Anfang des beiliegenden Assembler Programms.

Zeitgeber Der Inhalt des 32 Bit Registers mit der Adresse `TIMER` wird mit 100000 Schritten pro Sekunde automatisch hoch gezählt. Nach mehreren Stunden gibt es einen Overflow, so dass der Zähler wieder bei 0 beginnt. Das können Sie ignorieren.

Programmaufbau Unterteilen Sie das Programm in sinnvolle Unterprogramme, die mit vereinfachter Schnittstelle (s. Vorlesung) realisiert werden. Klare Schnittstellen vereinfachen die Tests, erhöhen die Übersichtlichkeit und führen schließlich zu einer stabileren Lösung und einer schnelleren Entwicklung. Typische Funktionen sind zum Beispiel

- **displaytime** aktualisiert die Zeitanzeige auf dem TFT-Display.
- **checktimer** liest den Zeitgeber aus und aktualisiert die Variable, die die Zeitspanne der Stoppuhr speichert.

Unterlagen, die vor dem Praktikum im Repo auf dem Server stehen müssen:

- Vor dem **siebten Praktikumstermin** sollten in der README.md Datei die zentralen Funktionen und Anforderungen sowie die Teilaufgaben aufgelistet werden. Im Prinzip ist das Ihrer Analyse der Aufgabe.

Weiterhin sollte `main.s` lauffähigen Beispiel Code zum Ansteuern der LEDs, zum Abfragen der Taster und zum Ansteuern des Displays enthalten. Der Code muss sinnvoll dokumentiert sein (Klasse statt Masse). Sie sollten entsprechende Assembler Unterprogramme entwerfen und umsetzen.

Arbeiten Sie als Starthilfe das beigefügte Assembler Programm durch.

- Vor dem **achten Praktikumstermin** sollte `main.s` lauffähigen Beispiel Code für die FSM und die Zeitmessung enthalten. Der Code muss sinnvoll dokumentiert sein (Klasse statt Masse).

Sie haben im Speicher eine oder mehrere Variablen angelegt, die die auszugebende Zeit auf dem Display speichern. Ein Assembler Unterprogramm liest diese Variablen aus und stellt die Uhrzeit auf dem Display dar.

- Vor dem **neunten Praktikumstermin** sollte `main.s` das fertige Assembler Programm der Stoppuhr enthalten. Der Code ist in sinnvoll gewählte Unterprogramme mit vereinfachter Schnittstelle unterteilt. Der Code muss sinnvoll dokumentiert sein (Klasse statt Masse).

Abnahme der Aufgabe im Praktikum Während der Abnahme im Praktikum müssen Sie Ihre Lösung darstellen und Änderungen an der Lösung vornehmen können.

Hinweise Lesen Sie sich die Hinweise in Ruhe durch. Sie tragen zum Verständnis der Aufgabe bei und helfen bei der Fehlersuche.

Aufteilung in Unterprogramme / Prozeduren Unterteilen Sie Ihr Programm in Unterprogramme / Prozeduren. Dazu folgende Hinweise:

- Kleine Unterprogramme, deren Funktion man in einen Satz beschreiben kann, sind gut. Auf seinem Abstraktionslevel erledigt ein Unterprogramm genau eine Aufgabe.
- In der Vorlesung wird u.a. eine sehr eingeschränkte Version für den Sprung in Unterprogramme dargestellt. Diese ist für die Aufgabe ausreichend. Hilfreich ist,

wenn Sie über die Register R0 bis R3 Argumente an ein Unterprogramm übergeben bzw. Ergebnisse vom Unterprogramm an das aufrufende Programm zurückgeben. Dokumentieren Sie für jedes Unterprogramm diese Ein- / Ausgabeschnittstelle.

- Der ISA Befehl `BL` speichert in Register `R14` (LR) die Rücksprungadresse – die Adresse zu der das Unterprogramm am Ende zurückspringt. Daher sind geschachtelte Unterprogrammaufrufe mit der eingeschränkten Schnittstelle von Unterprogrammen (s.o.) noch nicht möglich. Weiterhin darf ihr Assembler Programm `R14` nicht verändern – das macht nur der `BL` Befehl.

Beim Debuggen wird Ihnen auffallen, dass der **Debugger oftmals nicht beim ersten Befehl einer Assembler Prozedur anhält - sondern erst beim zweiten Befehl.**

- **Hintergrund** (können Sie überspringen): Ein Compiler, wie zum Beispiel ein C Compiler, fügt am Anfang und am Ende des Assembler Codes einer Funktion Maschinenbefehle ein, die den Stack verwalten. Diesen Code nennt man Prolog bzw. Epilog. Beim Debuggen auf Ebene der Hochsprache möchte man in der Regel, dass der Debugger im Prolog nicht anhält. Dazu generiert der Compiler entsprechenden Informationen für den Debugger (gdwarf-2). Bei einer Datei mit Assembler Code fehlt diese Information und somit schätzt der Debugger ab, wo der Prolog einer Funktion endet. Das führt dazu, dass man in einer *.s Datei in der Regel keinen Breakpoint auf den ersten Befehl einer Assembler Prozedur setzen kann bzw. StepInto erst beim zweiten Befehl der Assembler Prozedur hält.
- Es gibt drei Möglichkeiten, wie man trotzdem beim ersten Befehl einer Assembler Prozedur anhalten kann.
 - Man fügt als ersten Befehl der Assembler Prozedur einen NOP-Befehl ein. Der Debugger geht bei dem Befehl NOP davon aus, dass der Prolog fertig ist. Somit kann man auf den NOP-Befehl einen Breakpoint setzen.
 - Man setzt den Breakpoint im Disassembler View auf den ersten Befehl der Assembler Prozedur (typischer Weise ein PUSH-Befehl).
 - Man setzt einen Breakpoint bei dem BL-Befehl, der in die Assembler Prozedur springt. Über den Debugger Befehl stepi (Kurzform si) führt der Debugger genau eine Instruktion aus (daher das i wie Instruktion :) - also den BL-Befehl. Dann hält der Debugger am ersten Befehl der Assembler Prozedur an.
Leider gibt es für den Befehl si keinen Button im Debugger. Daher muss man in der Konsole des Debugger den Befehl „> si“ eingeben (das „>“ nicht vergessen).

Test Testen Sie Ihr Programm schrittweise. Das erleichtert den BringUp des Systems.

Genauigkeit der Stoppuhr Die Stoppuhr muss möglichst genau sein. **TIMER** arbeitet mit einer Genauigkeit von 10µs. Die auf dem TFT dargestellte Zeit hat eine Genauigkeit von 10ms. Verwenden Sie bei allen Zeitberechnungen Variablen, die die Zeiten mit

einer Genauigkeit von $10\mu\text{s}$ behandeln. Für die Ausgabe auf dem Display runden Sie dann auf eine Genauigkeit von 10ms.

Ausgabe der Zeit Intern wird die vergangene Zeitspanne in einer Variablen oder einem Register gespeichert. Die Einheit ist $10\mu\text{s}$. Sie können die auszugebenden Ziffern nicht einfach mit DIV und MOD-Operationen und entsprechenden 10-er Potenzen berechnen, da die Ausgabe Sekunden und Minuten darstellt und eine Minute 60 und nicht 100 Sekunden lang ist. Also müssen Sie die 10-er Potenzen durch andere Werte ersetzen.

Ein einfaches Verfahren zur Berechnung der vergangenen Zeit Der Timer aktualisiert mit der Frequenz 10^5 Hz das CNT-Register. Durch Lesen der Adresse TIMER erhält man den aktuellen Wert des CNT-Registers. Einen Wert dieses Registers nennt man Zeitstempel.

Eine Funktion UpdateClk speichert den aktuellen Zeitstempel und berechnet die Zeitspanne, die zwischen zwei Aufrufen der Funktion vergangen ist. Dies wird durch Subtraktion des Zeitstempels, der beim letzten Aufruf der Funktion gespeichert wurde, vom aktuellen Zeitstempel berechnet. Zur Initialisierung wird UpdateClk einmalig aufgerufen.

Am Anfang der SuperLoop wird UpdateClk aufgerufen. So erhält man die Zeitspanne, die im letzten Durchlauf der SuperLoop vergangen ist. Diese wird z.B. zur Zeitspanne der Stoppuhr addiert.

Super-Loop Sieht man von der Initialisierung ab, ist die äußere Struktur Ihres Programms eine **while (true)** Schleife. Diese nennt man Super-Loop. Der Rumpf der Super Loop kann zum Beispiel wie folgt strukturiert werden, so dass diese Schritte immer wieder in dieser Reihenfolge ausgeführt werden.

```
; Aktualisierung der gestoppten Zeitspanne

; lese Taster ein

; Update Zustand der Stoppuhr (INIT, RUNNING, HOLD)

; Wenn im Zustand INIT State: setze gestoppte Zeitspanne auf 0

; update LEDs in Abhängigkeit vom aktuellen Zustand

; Wenn nicht im Zustand HOLD: Uhrzeit auf dem TFT ausgeben
```